

Amazon ECS (Elastic Container Service) - Service Overview

Introduction

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that helps you deploy, manage, and scale containerized applications. ECS eliminates the need to install and operate your own container orchestration software, manage and scale a cluster of virtual machines, or schedule containers on those machines.

Core Concepts

- Cluster: a logical grouping of tasks or services. A cluster can run on EC2 instances that you manage, or on AWS Fargate, a serverless compute engine for containers.
- Task Definition: a text file (in JSON) that describes one or more containers that form your application, including the Docker image to use, CPU and memory requirements, networking mode, and IAM roles.
- Task: a running instantiation of a task definition, representing one or more containers running together.
- Service: maintains a specified number of running tasks simultaneously, automatically replacing any task that fails or stops, and can integrate with a load balancer to distribute traffic.

Launch Types

- Fargate: a serverless launch type where AWS manages the underlying compute infrastructure. You specify CPU and memory requirements, and AWS provisions and scales the infrastructure automatically. This removes the need to patch, scale, or secure EC2 instances directly.
- EC2: you manage a cluster of EC2 instances (the "container instances") on which ECS schedules your tasks, giving more control over instance types and cost optimization, at the cost of additional operational overhead.

Task Roles vs Execution Roles

ECS distinguishes between two IAM roles attached to a task:

- Task Execution Role: used by the ECS agent itself to pull the container image from ECR and write logs to CloudWatch Logs. This role does NOT grant permissions to the application code running inside the container.
- Task Role: assumed by the application code running inside the container, granting it permission to call other AWS services (e.g. reading from S3, writing to DynamoDB) using the AWS SDK.

Networking

With the `awsvpc` network mode (required for Fargate), each task receives its own elastic network interface (ENI) and private IP address within the VPC, giving task-level network isolation via security groups, similar to how an EC2 instance would be secured.

Service Auto Scaling

ECS Service Auto Scaling can automatically adjust the desired count of tasks in a service based on CloudWatch metrics such as CPU utilization, memory utilization, or a custom metric like request count per target from an Application Load Balancer.

Deployment Strategies

ECS supports rolling updates by default, and can also be configured for blue/green deployments via integration with AWS CodeDeploy, allowing traffic to shift gradually to a new task set while monitoring for errors before fully cutting over.

Logging

Containers typically use the `awslogs` log driver in their task definition, which streams container stdout/stderr directly to a specified CloudWatch Logs log group with no additional agent required inside the container.

Capacity Providers

A capacity provider determines the infrastructure that a task runs on. The `FARGATE` and `FARGATE_SPOT` capacity providers let ECS choose between standard Fargate pricing and discounted, interruptible Fargate Spot capacity, and can be combined in a capacity provider strategy that runs, for example, 70% of tasks on Fargate Spot for cost savings and 30% on standard Fargate for baseline reliability.

ECS vs EKS

Amazon Elastic Kubernetes Service (EKS) is AWS's managed Kubernetes offering, distinct from ECS. ECS uses AWS's own, simpler orchestration model and API, while EKS runs the open-source Kubernetes control plane, giving access to the broader Kubernetes ecosystem (Helm charts, Kubernetes-native tooling) at the cost of additional operational complexity. Teams already invested in Kubernetes tooling, or needing multi-cloud portability, typically choose EKS; teams wanting the simplest path to running containers on AWS with the least operational overhead typically choose ECS.

Common Use Cases

- Running containerized microservices and APIs without managing servers (via Fargate)
- Batch processing jobs
- Backing a CI/CD pipeline's deployment target for a Docker image built and pushed to ECR

Best Practices

- Use Fargate for workloads where operational simplicity outweighs the cost premium versus self-managed EC2 capacity.
- Keep the task execution role and task role separate and minimally scoped; never grant the execution role application-level permissions.
- Enable Container Insights for detailed CPU/memory/network metrics per task and service.
- Use the `awslogs` driver so structured JSON logs flow directly into CloudWatch without extra logging infrastructure.